

## TEII - Tarea Bloques 3 y 4

## Índice

|                                                                                                             |          |
|-------------------------------------------------------------------------------------------------------------|----------|
| <b>1. Tecnologías Específicas en Ingeniería Informática • Tarea Bloques 3 y 4</b>                           | <b>2</b> |
| 1.1. Instrucciones generales . . . . .                                                                      | 2        |
| 1.2. Apartado I . . . . .                                                                                   | 2        |
| 1.2.1. Fundamentos de Python e introducción al paquete <code>pandas</code> : <i>(2,00 puntos)</i> . . . . . | 2        |
| 1.3. Apartado II . . . . .                                                                                  | 3        |
| 1.3.1. El (sub)paquete <code>teii.finance</code> : <i>(5,00 puntos)</i> . . . . .                           | 3        |
| 1.4. Apartado III . . . . .                                                                                 | 4        |
| 1.4.1. Acceso a APIs HTTP con <code>marimo</code> : <i>(3,00 puntos)</i> . . . . .                          | 4        |
| 1.5. Entrega . . . . .                                                                                      | 4        |

# 1. Tecnologías Específicas en Ingeniería Informática • Tarea Bloques 3 y 4

## 1.1. Instrucciones generales

- Solo debe haber un repositorio (**teii-alumno**, o nombre similar) por cada grupo de prácticas. Podéis decidir vosotros la cuenta de *github* que usaréis para la entrega (puede ser la de cualquiera de los dos miembros del grupo), y compartirlo con el usuario de *github* **pedroe-um-es**.
- El código de partida que os hemos entregado (contenido en el archivo **teii-finance.2025-26.tgz** en el apartado de Recursos del aula virtual) se debe publicar en, al menos, un *commit* en dicho repositorio en *github* antes del **martes día 28 de abril a las 23:55h**. Los archivos contenidos en este commit se añadirán a los posibles que ya tengáis subidos (*notebooks* con ejercicios de python y pandas), y constituirán la base sobre la que se realizarán las modificaciones del apartado II de la entrega.
- El repositorio **debe** contener un archivo **GRUPO+REPO.md** con el nombre y apellidos de los miembros del grupo, además del URL completo del repositorio resultante en github.
- Para cada ejercicio se debe crear una rama aparte, llamada **Ejercicio [#]** donde [#] es la *keyword* correspondiente al mismo (ver apartados I, II y II más abajo). Dichas ramas se irán incorporando también **obligadamente** a la rama *main* mediante un *pull request*.
- Naturalmente, cada una de estas ramas podrá (y en algunos casos seguramente deberá) incluir varios commits que separen “semánticamente” el contenido de los mismos. Se valorará explícitamente esta estructuración en el desarrollo del trabajo.
- Además del seguimiento del procedimiento anterior, se valorará también explícitamente la debida separación por contenido y temporal entre los commits, y con continua interacción entre los miembros del grupo. En consecuencia, se penalizarán aquellas entregas que realicen todos estos ejercicios en bloque justo antes de la fecha límite de entrega.
- Cada miembro del grupo debe ser el autor de un porcentaje **razonable** de *pull requests* sobre el total de ejercicios (recordad que se intenta emular el trabajo en un equipo de desarrollo de producción real).
- Todos los métodos añadidos a la clase `TimeSeriesFinanceClient()` tienen que incluir *type annotations*.
- Todos los tests unitarios tienen que validar la salida de los métodos objeto de los tests.
- Si los métodos generan excepciones, se deben añadir también tests unitarios que comprueben y capturen la correcta generación de dichas excepciones (por ejemplo, `test_constructor_failure_invalid_api_key()`).
- Es también **imprescindible** que la ejecución de `tox`, sin hacer modificaciones al archivo `tox.ini`, finalice sin errores, incluyendo el testeo y *linting* completo en los entornos python 3.11, 3.12 y 3.13.
- Se debe publicar un *commit* final con el mensaje “*Entrega Final de Tarea*” en el repositorio en *github* antes de la fecha límite de entrega. No olvidéis tampoco que, además del repositorio *github*, se pide **explícitamente la entrega de dos archivos .zip** a través de la tarea correspondiente que se habilitará una semana antes de la entrega en el Aula Virtual (ver apartado Entrega).

## 1.2. Apartado I

### 1.2.1. Fundamentos de Python e introducción al paquete pandas: (2,00 puntos)

**Ejercicio [PYTHON]:** Realiza todos los ejercicios del *Jupyter notebook* Fundamentos de Python - Ejercicios.ipynb.

**Ejercicio [PANDAS]:** Realiza todos los ejercicios del *Jupyter notebook* Introduccion a Pandas - Ejercicios.ipynb.

## 1.3. Apartado II

### 1.3.1. El (sub)paquete `teii.finance`: (5,00 puntos)

**Ejercicio [CONSTRUCTOR]:** Escribe un test unitario llamado `test_constructor_env` que no dependa de `api_key_str` pero en el que el constructor también se ejecute con éxito. *Indicación:* Usa *monkey patching* con la variable de entorno `TEII_FINANCE_API_KEY`.

Añade también un test unitario llamado `test_constructor_unsuccessful_request()` en el que la llamada a `request.get` genere una excepción `requests.exceptions.ConnectionError`. Verifica que el constructor genera una excepción `FinanceClientAPIError`.

Finalmente, implementa también la comprobación de errores en el método `_build_data_frame()` de la clase `TimeSeriesFinanceClient()`, y escribe un test unitario llamado `test_constructor_invalid_data` que verifique que la comprobación de errores se realiza correctamente.

*Indicación:* Para esto último, crea un *ticker* `NODATA` y el correspondiente archivo `NODATA.json` en el lugar adecuado. Dicho archivo debe contener metadatos válidos pero NO datos semanales. Para poder usar el *ticker* `NODATA`, tendrás que extender la *fixture* `mocked_requests()`.

**Ejercicio [PRICE]:** Modifica la función `main()` de `example.py` para que la función `weekly_price()` sólo devuelva los datos disponibles para el año 2026. Implementa la comprobación de los parámetros `from_date` y `to_date` en el método `weekly_price()` de la clase `TimeSeriesFinanceClient()`, y escribe un test unitario llamado `test_weekly_price_invalid_dates` que verifique que la comprobación de errores se realiza correctamente.

**Ejercicio [VOLUME]:** Escribe tests unitarios para el método `weekly_volume()` de la clase `TimeSeriesFinanceClient()` similares a los del método `weekly_price()`. La salida debería ser similar a la proporcionada en los archivos `TIME_SERIES_WEEKLY_ADJUSTED.NVDA.volume.unfiltered.csv` y `TIME_SERIES_WEEKLY_ADJUSTED.NVDA.volume.filtered.csv` del subpaquete `teii.finance.data`.

**Ejercicio [DIVIDENDS]:** Implementa el método `yearly_dividends(from_year, to_year)` (los parámetros serán enteros y opcionales) de la clase `TimeSeriesFinanceClient()`. Este método tiene que calcular el dividendo total anual del *ticker* elegido.

Escribe tests unitarios para dicho método. Los tests deben verificar que el resultado es correcto, apoyándose en los archivos `TIME_SERIES_WEEKLY_ADJUSTED.IBM.dividend.[un]filtered.csv` para ello. Siempre pueden añadirse archivos a `teii.finance.data` si se considera necesario.

**Ejercicio [VARIATION]:** Implementa el método `highest_weekly_variation(from_date, to_date)` (los parámetros son fechas opcionales) de la clase `TimeSeriesFinanceClient()`. Este método tiene que calcular la fecha en la que se produjo una mayor variación de la cotización del *ticker*, es decir, la fecha que maximiza `high-low`. Para esa fecha, el método debe devolver una tupla (`fecha`, `high`, `low`, `high-low`) donde `fecha` será de tipo `datetime.date`, mientras que los restantes valores serán `float`.

Escribe tests unitarios para dicho método. Los tests deben verificar que el resultado devuelto es correcto, para algún *ticker* particular, y tanto para una llamada sin intervalo de fechas, como para un intervalo limitado mínimo (de nuevo, apoyándose en datos contenidos en `teii.finance.data`).

**Ejercicio [LOGGING]:** Añade al menos un mensaje de *logging* a cada uno de los métodos de la clase `TimeSeriesFinanceClient()`, tanto existentes como los que hayas desarrollado. Realiza después las modificaciones pertinentes en el script `example.py` para que todos los mensajes de *logging* se envíen a un archivo llamado `example.log`. Realiza un primer commit usando el método genérico `logging.basicConfig`, y otro posterior modificando el anterior basado en la configuración más explícita del objeto `logger` correspondiente. Verifica el comportamiento correcto asignando diferentes niveles de *logging* tanto para `example.py` como para `teii.finance`.

**Ejercicio [DOC]:** Documenta el paquete `teii`, el subpaquete `teii.finance` y todos sus módulos siguiendo el formato NumPy/SciPy docstrings.

**Ejercicio [CICD]:** Modifica el archivo de configuración de GitHub Actions `teii-cicd.yaml` para que los tests se ejecuten para *todas* las combinaciones a partir de las siguientes versiones de Python y sistema operativo:

- `py311`, `py312` y `py313`.
- `ubuntu-22.04` y `ubuntu-24.04`.

**Nota:** El segundo *job* que publica el paquete `.whl` a TestPyPi no es obligado, es decir, se puede dejar comentado, aunque se valorará también su correcta inclusión.

## 1.4. Apartado III

### 1.4.1. Acceso a APIs HTTP con `marimo`: (3,00 puntos)

**Ejercicio [MARIMO]:** Realizar un *notebook* reactivo y autocontenido con `marimo` que consuma datos de una API HTTP pública y los procese con `pandas`, generando al menos una tabla y algún gráfico interactivo (con libertad total sobre qué datos mostrar y cómo presentarlos). Se evaluará que se demuestren los fundamentos de *notebooks* reactivos con `marimo`, el manejo correcto de `pandas` y alguna librería gráfica (`matplotlib`, `plotly`, `altair` o similar), y el acceso a APIs HTTP.

Podéis usar, por ejemplo, Open-Meteo, incluyendo su API de geocodificación u otra API pública similar. Pero se permite elegir cualquier API distinta que os llame la atención, siempre que sea de acceso sencillo y documentado.

Usando una filosofía similar a la de `teii.finance.data`, el *notebook* debe funcionar por defecto con algunos datos predescargados locales (**que se incluirán en la entrega**), pero debe ofrecer además una opción alternativa para consultar directamente la API original de la que provienen esos datos (para evitar posibles limitaciones de acceso, cuota o disponibilidad).

## 1.5. Entrega

- La fecha límite de entrega de esta tarea es el **martes 19 de mayo de 2026 a las 23:55h**.
- Cread un archivo llamado `TEII-finance - Nombre1 Apellidos1 - Nombre2 Apellidos2.zip` con las siguientes órdenes CLI:  

```
$ cd teii-alumno  
$ git archive -o "TEII-finance - Nombre1 Apellidos1 - Nombre2 Apellidos2.zip" HEAD
```
- Cread también otro **archivo aparte** llamado `TEII-notebooks - Nombre1 Apellido1 - Nombre2 Apellido2.zip` que contenga tanto los dos Jupyter *notebooks* como el *notebook* de `marimo` solicitados en los apartados I y III (independientemente de que éstos aparezcan también en el repositorio git entregado).

Subid ambos archivos al Aula Virtual antes de la fecha límite indicada. No olvidéis la correcta cumplimentación del archivo `GRUPO+REPO.md` mencionado en las Instrucciones generales .